

Functions as values: iterators and streams

Block 2 — Thunks and streams: when is a value computed?

SW7 · Aalborg University Copenhagen · Autumn 2026

Block 2: the three parts

In a strict language a value is computed where it is written. In this block we put a **function** in its place.

- | | | |
|---|---------------------------------------|---|
| 1 | Thunks
when? | We choose when the value is computed: when we call the function. |
| 2 | Sequences
how far? | We put the function inside a data structure. We choose how far a sequence is computed, and it can be without end. |
| 3 | Memoisation
how many times? | We keep the result of the function. We choose that the value is computed at most once. |

Thunks

How does a strict language wait?

Session 4: if written as a function

```
let my_if c t e = if c then t else e
```

OCAML

```
let rec fact n =  
  my_if (n = 0) 1 (n * fact (n - 1))
```

`fact 5` never returns. OCaml computes the two branches before it calls `my_if`.

```
let my_if c t e = if c then t () else e ()
```

OCAML

```
let rec fact n =  
  my_if (n = 0) (fun () -> 1)  
    (fun () -> n * fact (n - 1))
```

`fact 5` is 120. `my_if` calls one of the two functions.

A function that holds a computation and waits for its call is a **thunk**. In OCaml we write it `fun () -> e`.

A thunk, called twice

```
const t = () => {  
  console.log("computing");  
  return 6 * 7;  
};
```

JAVASCRIPT

```
console.log("the thunk is made");  
const a = t();  
const b = t();  
console.log(a, b);
```

Run

Reset

```
the thunk is made  
computing  
computing  
42 42
```

What does the program print, and in which order?

- When the thunk is made, its body is not run.
- At every call the body is run again.

The same program in OCaml

```
let t = fun () ->  
  print_endline "computing";  
  6 * 7
```

OCaml ↗

```
let () = print_endline "the thunk is made"  
let a = t ()  
let b = t ()
```

A thunk computes **at every call**. It is session 4's **call-by-name**, written by the program.

A left fold and a right fold: who holds the rest of the list?

```
let rec fold_left f acc xs =  
  match xs with  
  | [] -> acc  
  | x :: xs -> fold_left f (f acc x) xs
```

OCaml

Our function receives the accumulator and **one element**. The rest of the list, `xs`, stays with the fold.

```
let rec fold_right f xs z =  
  match xs with  
  | [] -> z  
  | x :: xs -> f x (fold_right f xs z)
```

OCaml

Our function receives one element and **the fold of the rest**, as its second argument.

Only with a **right fold** is our function given the rest, and can decide what becomes of it.

`fold_right` keeps one frame waiting per element; `fold_left` keeps none: its recursive call is a tail call.

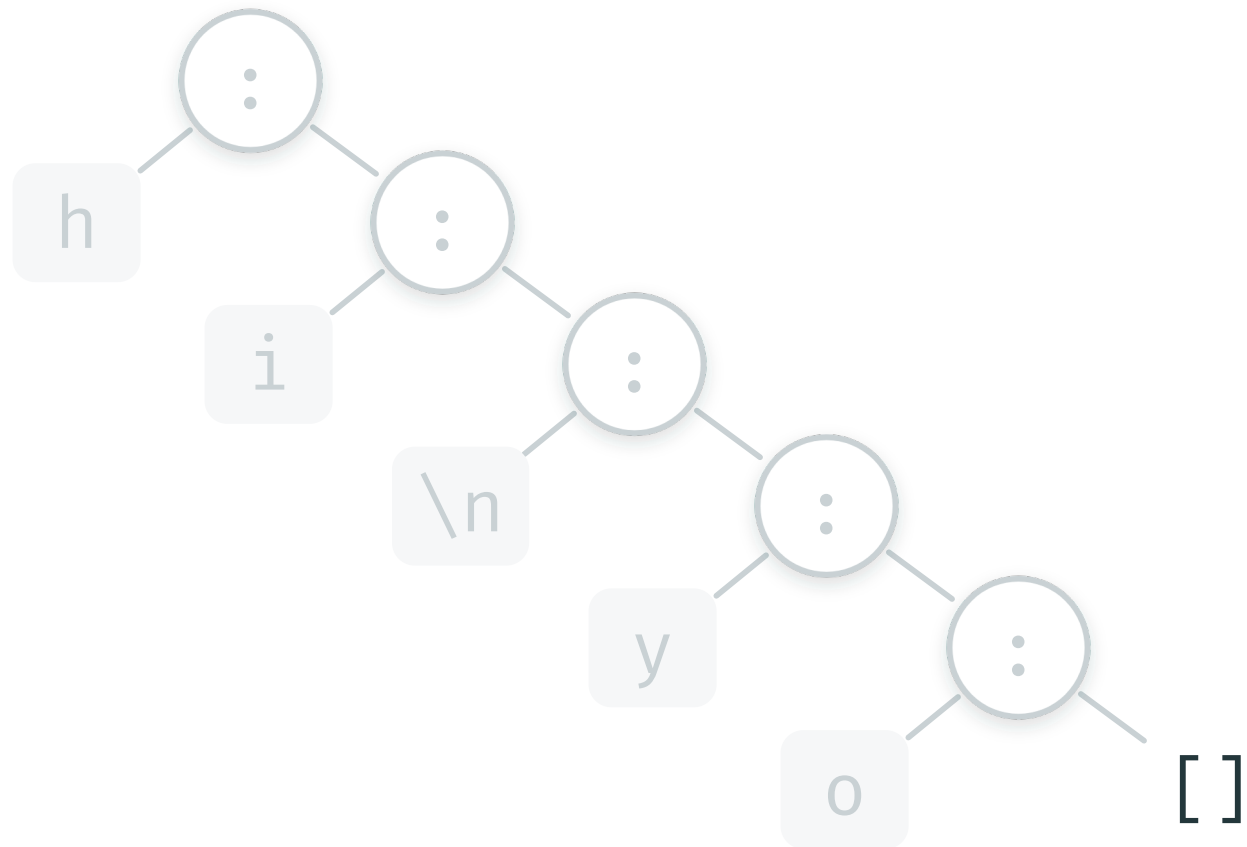
Block 1's exists, with the left fold

```
let exists p l = fold_left (fun found x -> found || p x) false l
```

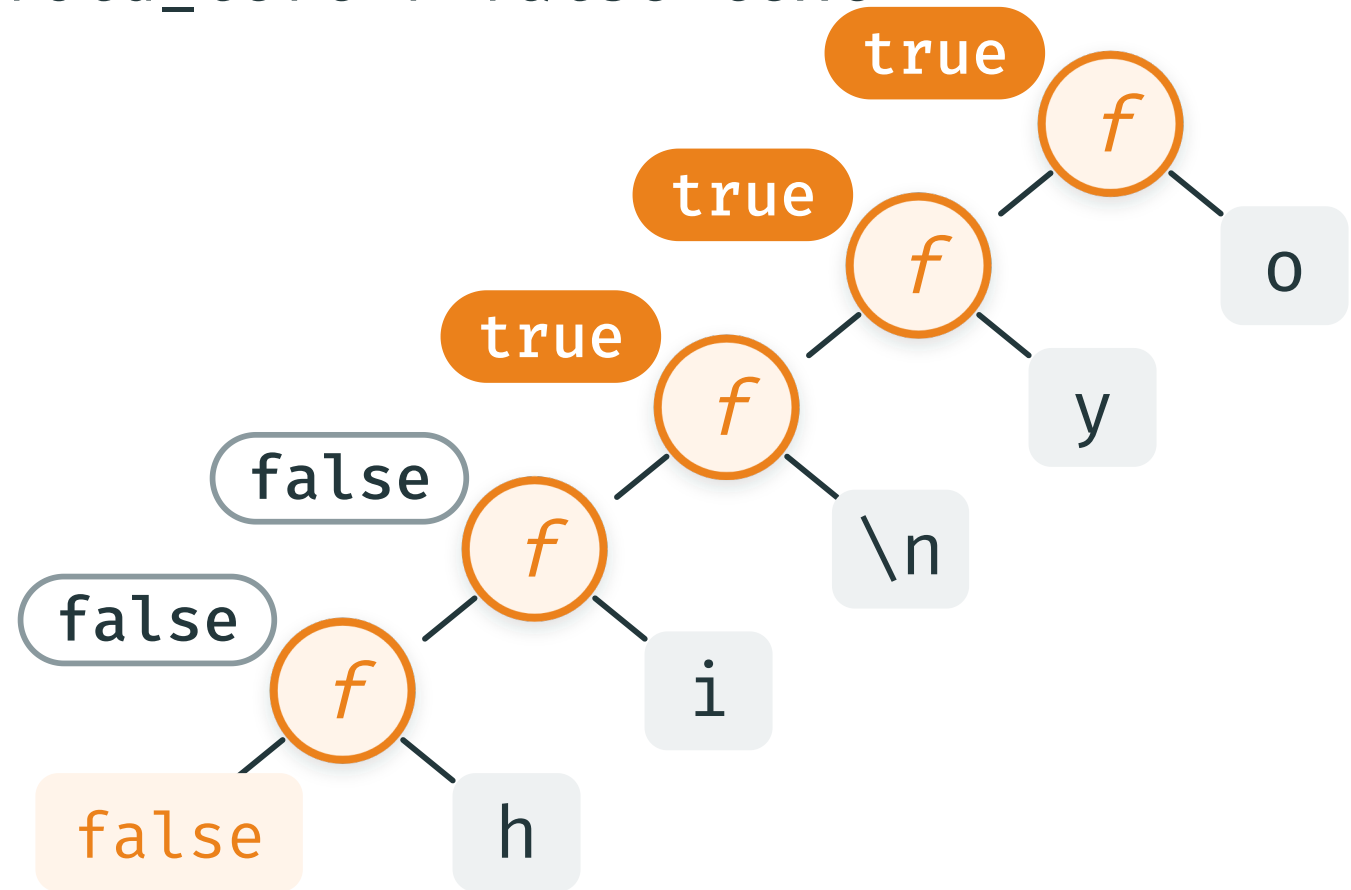
[OCaml ↗](#)

```
exists (fun c -> c = '\n') text
```

the list



fold_left f false text



Step Run Reset

f called 5 times

f was given one element at a time. The rest of the list never reached it: 5 calls

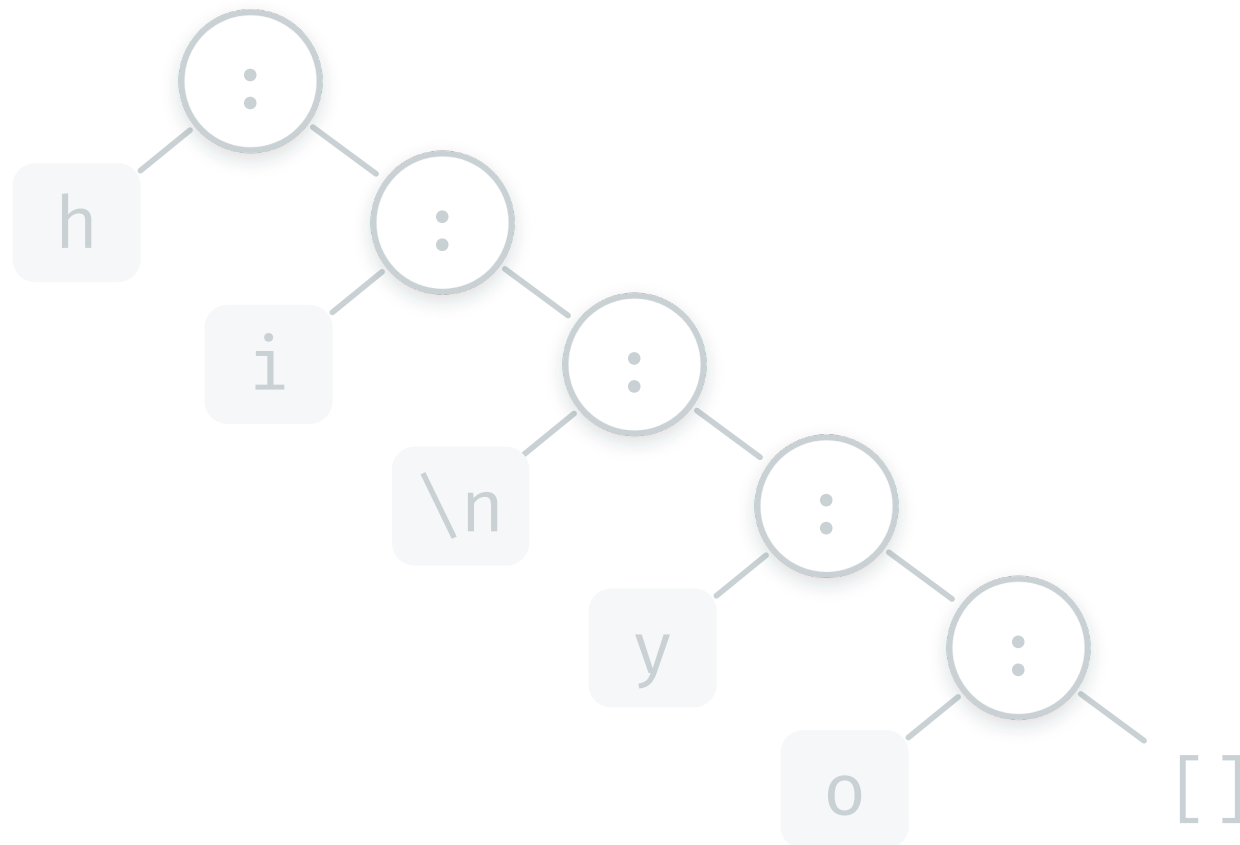
exists, with the right fold

```
let exists p l = fold_right (fun x rest -> p x || rest) l false
```

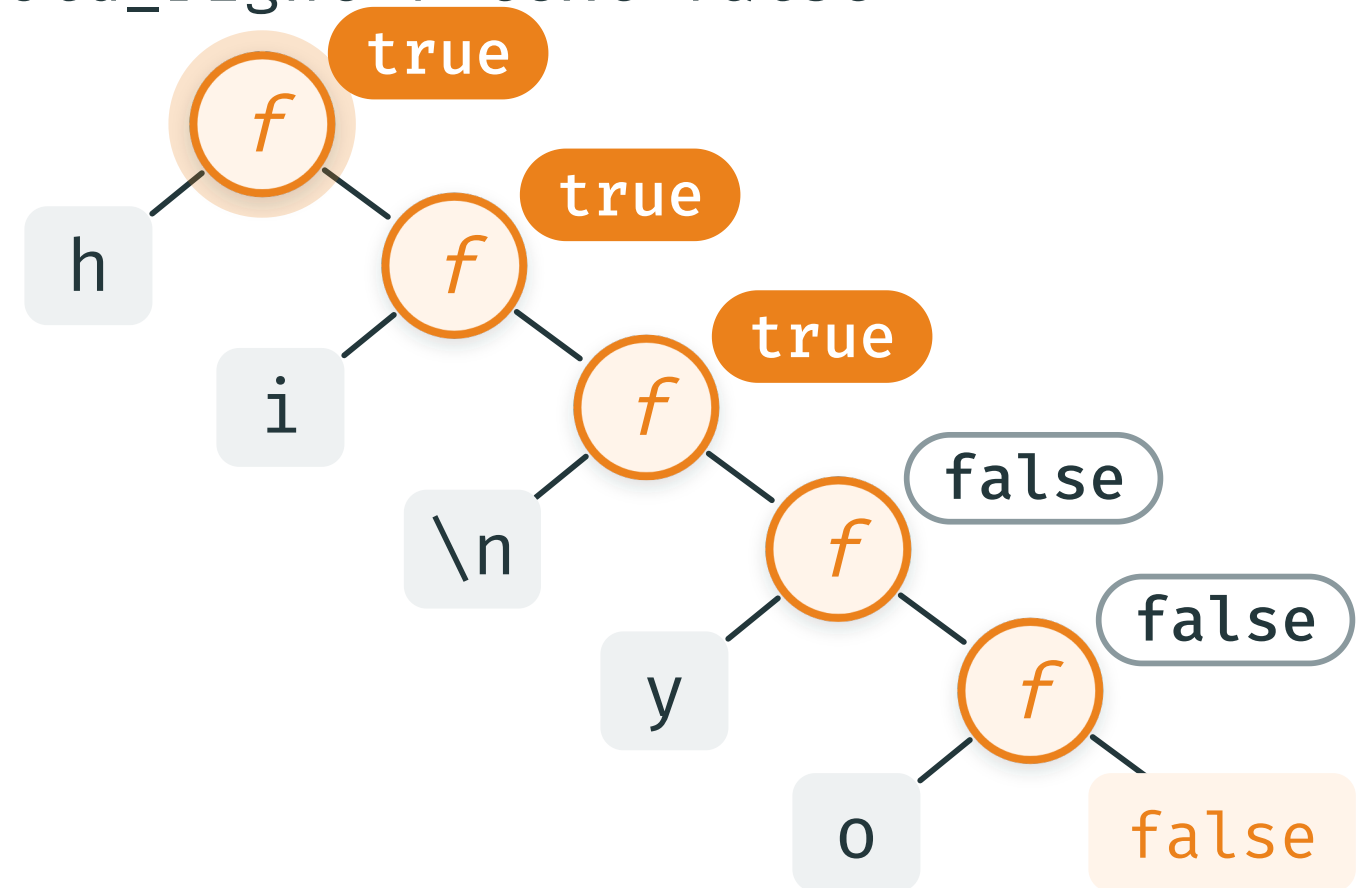
[OCaml ↗](#)

```
exists (fun c -> c = '\n') text
```

the list



fold_right f text false



Step Run Reset

f called 5 times

f is called on 'h'. Its second argument is true, already computed: it answers true

The right fold, with the rest as a thunk

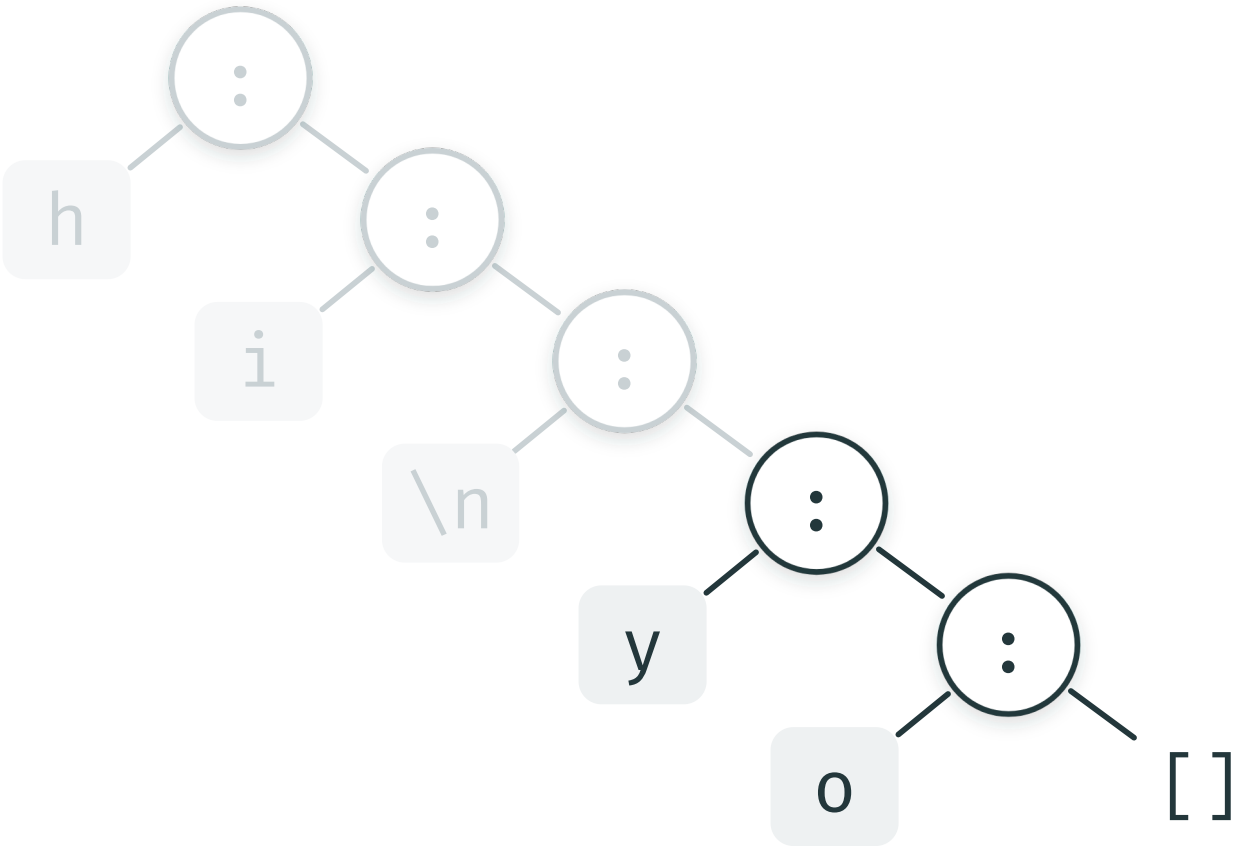
```
let rec fold_right f xs z = match xs with
| [] -> z
| x :: xs ->
    f x (fun () -> fold_right f xs z)
```

OCaml

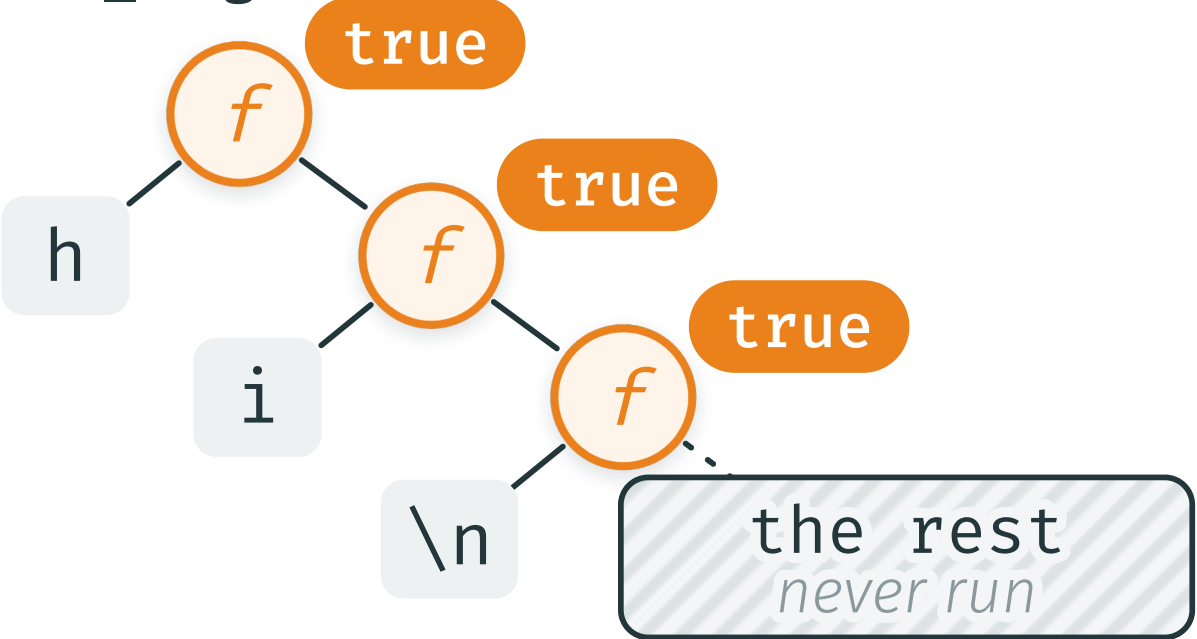
```
let exists p l =
    fold_right
      (fun x rest -> p x || rest ())
      l false
```

OCaml ↗

the list



fold_right f text false



The list of **all** the natural numbers

```
let rec from n = n :: from (n + 1)
```

OCaml ↗

```
let rec take n l =  
  if n = 0 then []  
  else match l with  
    | [] -> []  
    | x :: rest -> x :: take (n - 1) rest
```

```
take 5 (from 0)
```

Stack overflow during evaluation

```
from n = n : from (n + 1)
```

Haskell ↗

```
take n l =  
  if n == 0 then []  
  else case l of  
    [] -> []  
    x : rest -> x : take (n - 1) rest
```

```
take 5 (from 0)
```

[0,1,2,3,4]

The two programs are the same, line for line. Why does Haskell give an answer, and OCaml none?

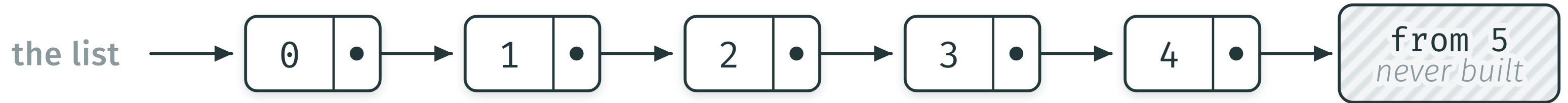
A list built **lazily**: when it is read

```
take 5 (from 0)
```

```
-- Haskell also writes from 0 as [0 ..]
```

HASKELL

take 0



Step Run Reset

cells built 5 cells in the list **no end**

take 0 reads nothing: the program ends. The rest of the list, from 5, is never built

Haskell builds a cell of the list when a function **reads** it, and not before. `take 5` reads five cells, so five cells are built.

The list of all the prime numbers

```
primes = sieve [2 ..]
  where sieve (p : xs) = p : sieve [x | x <- xs, x `mod` p /= 0]
```

[Haskell ↗](#)

in Haskell

we read

as we know it

[2 ..]

the list 2, 3, 4, ... without end

from 2

p : xs

a list: its first element `p`, its rest `xs`

OCaml: `p :: xs`

[x | x <- xs, c]

the list of the `x` of `xs` for which `c` holds

Python: `[x for x in xs if c]`

x `mod` p /= 0

`x` is not a multiple of `p`

OCaml: `x mod p <> 0`

where

a definition written after its use

OCaml: `let ... in`, before

`sieve` keeps `p`, the first element, and calls itself on the rest **without the multiples of** `p`. The list in brackets is block 1's `filter`.

take k primes: how far is [2 ..] built?

```
take 10 primes = [2,3,5,7,11,13,17,19,23,29]
```

The sieve is applied to a list without end, and it calls itself on a list without end. Why does the program end?

take 1 primes	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...
take 2 primes	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...
take 3 primes	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...
take 4 primes	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...
take 5 primes	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	...

A number is struck out when a prime kept before it divides it. To give its first k primes, Haskell builds `[2 ..]` up to the k -th prime: the hatched numbers are **not yet computed**.

Lazy evaluation, in Haskell

In Haskell no argument is computed before the call. It is computed when the function uses it, and at most once: session 4's **call-by-need**.

in OCaml, we write

in Haskell

Block 1	another fold, to stop before the end of a list	the right fold stops by itself
Block 2	a thunk for each value that must wait	every value waits, and is computed once
Session 6	two functions, for a search that can fail	the list of all the solutions, of which we read the first

In session 4 the language chose when an argument is computed. In session 5 **the program chooses**, with a function.

The two folds, on a list without end

```
foldr (\x rest -> x > 2 || rest)
      False [0 ..]
```

Haskell ↗

True

```
foldr f False [0 ..]
= f 0 (foldr f False [1 ..])
= ...
= f 3 (foldr f False [4 ..])
= True
```

HASKELL

The call on top is `f`, our function. At 3 it answers, and does not read its rest.

```
foldl (\found x -> found || x > 2)
      False [0 ..]
```

HASKELL

the program does not end

```
foldl f False [0 ..]
= foldl f (f False 0) [1 ..]
= foldl f (f (f False 0) 1) [2 ..]
= foldl f (f (f (f False 0) 1) 2) [3 ..]
= ...
```

HASKELL

The call on top is always `foldl`: no step answers. The calls of `f` wait, and are never run.

Laziness delays what a function is given. In Haskell as in OCaml, only with the **right fold** is our function given the rest.

Sequences

How do we postpone the next element?

From a list to a sequence

```
type 'a list =  
  | []  
  | (::) of 'a * 'a list
```

OCaml

The rest of a list is a list: a **value**. It is computed before we read the first element.

```
type 'a node =  
  | Nil  
  | Cons of 'a * 'a seq
```

OCaml

```
and 'a seq = unit -> 'a node
```

The rest of a sequence is a **thunk**. It is computed when we call it.

A **sequence** (also called a stream)* is a thunk that gives one element and the rest of the sequence.

* CS3110, section 9.4. OCaml's library has this type as `Seq.t`.

nats and take

```
let rec nats n : int seq =  
  fun () -> Cons (n, nats (n + 1))
```

OCaml ↗

```
let rec take n (s : 'a seq) =  
  if n = 0 then []  
  else match s () with  
    | Nil -> []  
    | Cons (x, rest) ->  
      x :: take (n - 1) rest
```

`nats 0` is the sequence of all the natural numbers.

What does `take 5 (nats 0)` compute, and how many elements of `nats 0`?

`[0; 1; 2; 3; 4]` `produced=5`

A sequence without end is a **value**: we can name it, pass it and keep it. Five of its elements are computed, because we ask for five.

A list and a sequence, drawn

a list: every cell is built before we read the first one



a sequence: a cell is built when we call the thunk



Step **Run** **Reset**

cells built **3**

it builds one cell: the number 2, and the thunk nats 3

After three calls, three cells are built. The rest, `nats 3`, is a thunk that **nobody has called**.

map and filter on a sequence

```
let rec map f (s : 'a seq) : 'b seq =  
  fun () -> match s () with  
    | Nil -> Nil  
    | Cons (x, rest) ->  
      Cons (f x, map f rest)
```

OCaml ↗

`map f s` returns at once: its body is under
`fun () -> .`

In a `Cons`, the recursive call returns at once: it makes a function that **nobody has called yet**. The rest is computed when the consumer asks for it.

```
let rec filter p (s : 'a seq) : 'a seq =  
  fun () -> match s () with  
    | Nil -> Nil  
    | Cons (x, rest) ->  
      if p x  
      then Cons (x, filter p rest)  
      else filter p rest ()
```

OCaml

When `x` does not pass, `filter` asks for the next element itself.

How far is a sequence computed?

```
let even n = n mod 2 = 0
let square n = n * n

let squares =
  map square (filter even (nats 0))
```

OCaml ↗

```
produced=0
```

```
take 5 squares
```

OCAML

```
[0; 4; 16; 36; 64]    produced=9
```

How many elements has `nats 0` produced when `squares` is defined? And after `take 5`?

Nine numbers, from 0 to 8. Five of them are even.

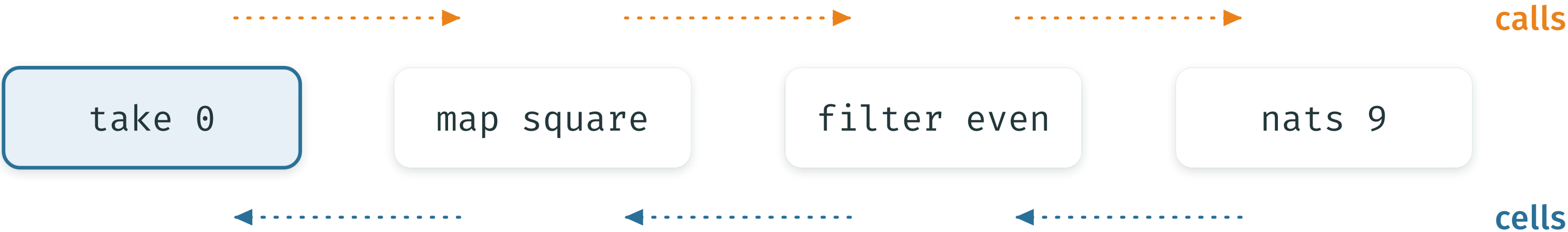
Defining `squares` computes no element. The consumer decides **how far** it is computed.

take 5 squares, step by step

```
= [0; 4; 16; 36; 64]
```

the consumer

the producer



Step Run Reset

take 0 is the empty list: take calls nothing more, and nats 9 is never called

cells built by nats 9

Stopping, by not asking

```
let rec exists p (s : 'a seq) =  
  match s () with  
  | Nil -> false  
  | Cons (x, rest) ->  
    p x || exists p rest
```

OCaml ↗

The text of block 1, `hi\nyo`, as a sequence of characters:

```
exists (fun c -> c = '\n') text
```

OCAML

```
found=true asked=3
```

With a sequence **we pull**. To stop, we do not ask for the next element.

The library: Seq

```
Seq.ints 0
|> Seq.filter (fun n -> n mod 2 = 0)
|> Seq.map (fun n -> n * n)
|> Seq.take 5
|> List.of_seq
```

[OCaml ↗](#)

```
[0; 4; 16; 36; 64]
```

`Seq.t` is the type that we wrote, and `Seq.ints 0` is our `nats 0`.

`x |> f` is `f x`. With it the line reads in the order the elements travel, from `ints` to `take`, and not from the inside out.

Memoisation

Can a value be computed at most once?

A thunk and a lazy value, each asked twice

```
let ticks = ref 0
```

OCaml ↗

```
let t = fun () -> incr ticks; 6 * 7
```

```
let a = t ()
```

```
let b = t ()
```

a=42 b=42 ticks=2

```
let ticks = ref 0
```

OCAML

```
let l = lazy (incr ticks; 6 * 7)
```

```
let a = Lazy.force l
```

```
let b = Lazy.force l
```

a=42 b=42 ticks=1

What is `!ticks` at the end of each program?

A thunk **delays**. A lazy value delays and **remembers**: it is session 4's **call-by-need**.

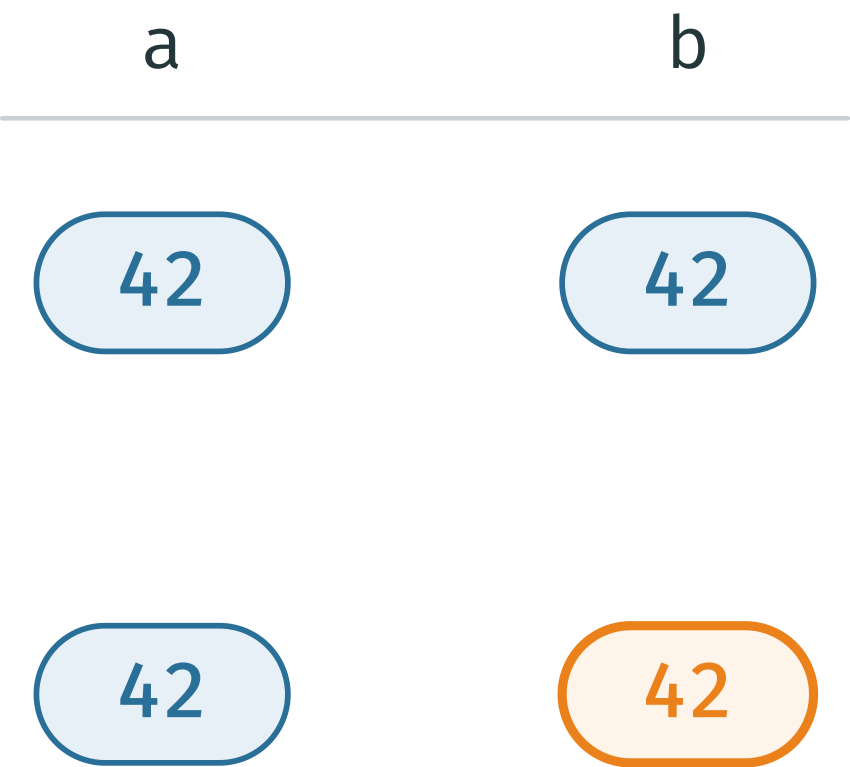
Okasaki 1998, chapter 4: a *stream* is part 2's sequence with a lazy value, not a thunk, for the rest.

A thunk and a lazy value, drawn

a thunk



a lazy value



Step Run Reset

b = Lazy.force l: 42 is read. Nothing runs

the body of the thunk has run 2 of the lazy value 1

lazy, made by hand

```
type 'a state =  
  | Delayed of (unit -> 'a)  
  | Value of 'a  
  
let delay f = ref (Delayed f)  
  
let force s = match !s with  
  | Value v -> v  
  | Delayed f ->  
    let v = f () in  
    s := Value v;  
    v
```

OCaml ↗

A lazy value is a reference that holds a thunk, and after the first `force` the value.

```
let s = delay (fun () ->  
  incr ticks; 6 * 7)  
  
let a = force s  
let b = force s
```

OCAML

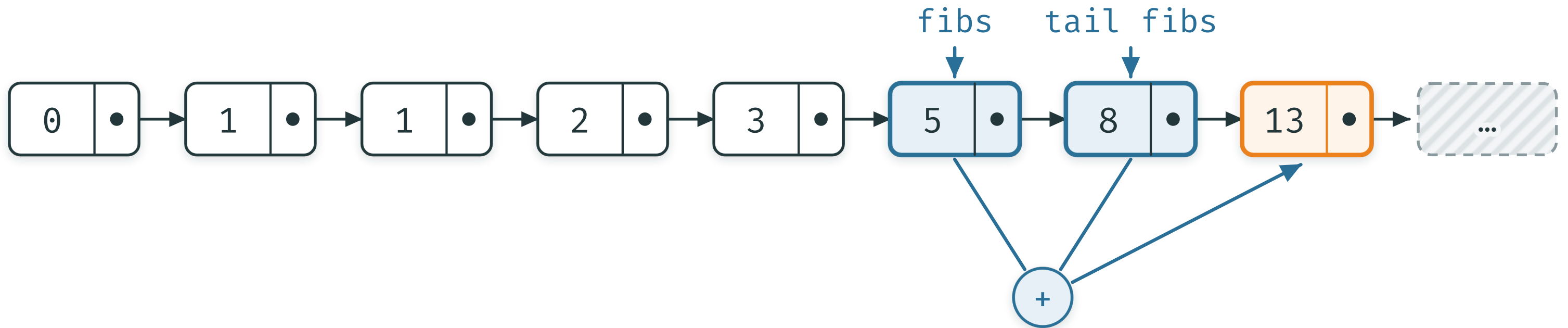
```
a=42 b=42 ticks=1
```

With `delay` we write the `fun () ->` ourselves. `lazy e` is a **keyword**: a function would receive `e` already computed.

With a table in the place of the reference, we keep one result for each argument. This is the **memoisation** of a function: see the appendix.

The Fibonacci numbers, as a list without end

```
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

[Haskell ↗](#)

[Step](#) [Run](#) [Reset](#)

additions, every cell kept **6** every cell computed again **46**

cell 7 is asked for. The two cells it reads are already there: one addition, $5 + 8 = 13$

The list is read at two places, one cell apart, and their sum is its next cell. Each cell is computed **once**, and read twice.

fibs: how many additions?

```
type 'a node = Nil | Cons of 'a * 'a seq
and 'a seq = 'a node Lazy.t
(* zip_with and tail read their sequences with Lazy.force *)
let rec fibs = lazy (Cons (0, lazy (Cons (1, zip_with ( + ) fibs (tail fibs)))))
```

[OCaml ↗](#)

The same list in OCaml. To compute its first 25 numbers, how many additions are needed?

the rest of the sequence is

additions

a thunk, in OCaml

196 392

a `lazy` value, in OCaml

23

a list, in Haskell

23

With `lazy`, `fibs` and `tail fibs` **share** the numbers. In Haskell every value is lazy, by default.

Lab 2's queue, without memoisation

```
let pop = function
  | x :: front, rear ->
    Some (x, (front, rear))
  | [], rear ->
    match rev rear with
    | [] -> None
    | x :: front ->
      Some (x, (front, []))
```

OCaml ↗

Lab 2's queue, as we wrote it there: a front and a rear. When the front is empty, `pop` reverses the rear.

```
(* q: eight elements pushed *)
let a = pop q
let b = pop q
```

OCAML

```
a=1 b=1 reversals=2
```

The queue is persistent: `q` can be used twice. With this `pop`, the reversal is then paid twice.

With `lazy` the reversal is suspended inside `q`: the two pops share it, and it is **paid once**. We write that queue as homework, at the end of the warm-up's sheet.

Source: Okasaki 1998, chapter 6, the banker's queue.

Conclusion of block 2

	the question	the program decides, in OCaml	the language decides, in Haskell
1	When is a value computed?	When we call the thunk that holds it.	When it is used.
2	How far is a sequence computed?	As far as we ask: a sequence .	Every list is a sequence.
3	How many times?	Once, if we keep the result: <code>lazy</code> .	Once: every value is lazy, by default.

In this block a function held **a computation not yet run**: `unit -> 'a`. We call it to get a value.

In session 6 a function holds **the rest of the computation**: `'a -> 'r`. We call it to give a value.

Session 6: a promise is a value that is not there yet, together with what to do when it arrives.

Memoising a function

Can a function remember what it has computed?

fib 25, without memoisation

```
let rec fib n =  
  if n < 2 then n  
  else fib (n - 1) + fib (n - 2)
```

```
let a = fib 25  
let b = fib 25
```

OCaml ↗

a=75025 b=75025 the body ran 485570 times

The second call computes everything again: a function keeps nothing from one call to the next. With `lazy (fib 25)` we keep one result, for the argument 25. Every other argument needs a lazy value of its own.

A lazy value keeps the result of **one computation**. To memoise a function is to keep a result **for each argument**: a table in the place of the reference.

memo: a function that keeps a table

```
let memo f =  
  let table = Hashtbl.create 16 in  
  fun x ->  
    match Hashtbl.find_opt table x with  
    | Some y -> y  
    | None ->  
      let y = f x in  
      Hashtbl.add table x y;  
      y
```

OCaml ↗

`memo f` makes a table, once, and returns a function.

- Called on `x`, the function looks `x` up in the table.
- If the result is there, it returns it, and `f` is not called.
- If not, it calls `f`, keeps the result, and returns it.

The function returned is a **closure with a state**, as block 1's counter. `lazy` is the same with one entry: the table of a function of no argument.

memo fib: the calls that the table sees

```
let rec fib n =  
  if n < 2 then n  
  else fib (n - 1) + fib (n - 2)  
  
let mfib = memo fib
```

[OCaml ↗](#)

	result	the body of <code>fib</code> ran
<code>mfib 25</code>	75025	242 785 times
<code>mfib 25</code> , again	75025	0 times

The second call is answered from the table. The first one is as slow as `fib 25`.

Inside `fib`, the two recursive calls are calls of `fib`, not of `mfib`. The table sees only the calls that **we** make.

memo_rec: the function receives the function to call

```
let fib' self n =  
  if n < 2 then n  
  else self (n - 1) + self (n - 2)
```

OCaml

`fib'` does not call itself: it has no `rec`. It calls `self`, which it receives.

```
memo_rec fib' 25 = 75025  
the body ran 26 times
```

```
let memo_rec f =  
  let table = Hashtbl.create 16 in  
  let rec g x =  
    match Hashtbl.find_opt table x with  
    | Some y -> y  
    | None ->  
      let y = f g x in  
      Hashtbl.add table x y;  
      y  
  in  
  g
```

OCaml ↗

`memo_rec` hands `g`, the function with the table, to `f` as its `self`. Every call goes **through the table**.

Functions of order 1, 2 and 3

	its type	order	
<code>fib</code>	<code>int -> int</code>	1	its argument is not a function
<code>memo</code>	<code>('a -> 'b) -> ('a -> 'b)</code>	2	its argument is a function
<code>fib'</code>	<code>(int -> int) -> (int -> int)</code>	2	
<code>memo_rec</code>	<code>(('a -> 'b) -> ('a -> 'b)) -> ('a -> 'b)</code>	3	its argument is a function whose argument is a function

The three ways to keep a result are one construction: a reference for **one value** (`lazy`), a list for **each position** (`fibs`), a table for **each argument** (`memo`).

Two exercises

1. Given `memo`, write `delay` and `force` of the slide “`lazy`”, made by hand” with it, and count the ticks.
2. Given `memo_rec` and the numbers of Pascal's triangle,

```
let rec choose (n, k) =  
  if k = 0 || k = n then 1  
  else choose (n - 1, k - 1) + choose (n - 1, k)
```

OCaml

write `choose'`, which receives the function to call, and count how many times each body runs for `(24, 12)`.

	result	the body ran
<code>choose (24, 12)</code>	2 704 156	5 408 311 times
<code>memo_rec choose' (24, 12)</code>	2 704 156	168 times